

4 차 프로젝트

기획서

AI 기반 지능형 교통·차량

통합 관제 시스템

차량 관제 · 경로 탐색 · 영상 분석 · 운행 기록

팀원

개발 기간

제출일

1. 프로젝트 개요

1.1 프로젝트명

AI 기반 지능형 교통·차량 통합 관제 시스템

1.2 프로젝트 소개

부산 지역의 차량 위치, 계획 경로, 차량 카메라 영상과 운행 기록을 하나의 웹 관제 화면에서 확인하는 시스템을 개발한다. 차량 제원을 반영한 경로 탐색과 가상 배차·도로 통제 시뮬레이션을 연결하고, 영상에서는 Custom YOLO26-seg 로 객체를 분할하며 UniDepth 로 거리를 측정한다.

1.3 개발 배경

차량 운영에서는 지도상의 위치만으로 도로 상황을 충분히 파악하기 어렵다. 카메라 영상, 경로와 이동 궤적, 과거 운행 기록을 함께 확인할 수 있어야 한다. 특히 대형 차량은 높이·중량·너비와 회전 제한을 반영한 경로가 필요하며, 도로 정체나 통제에 따른 대응을 실제 운행과 분리된 환경에서 검토할 수 있어야 한다.

1.4 프로젝트 목표

- 여러 출처의 차량 위치와 경로를 통합 지도에서 제공한다.
- Android 영상과 객체 분할·거리 정보를 관제 화면에 연결한다.
- 녹화 영상·탐지 결과·GPS 를 같은 시간축으로 재생한다.
- 가상 차량의 배차, 경유지 변경과 통제 상황의 우회를 시연한다.

표 1. 주요 사용자와 역할

사용자	역할	관심사
관리자	계정·차량 관리, 권한·설정 관리	데이터 적합성과 접근 통제
관제 운영자	차량·경로·영상 관제, 운행·가상 배차	상황 파악과 대응
조회자	지도·영상·운행 기록 조회	읽기 전용 정보 확인
Android 단말	배정 운행 확인, 카메라·GPS/IMU 송출	운행과 송출의 안정성

2. 요구사항 분석

2.1 기능 요구사항

표 2. 기능 영역별 요구사항

영역	주요 기능	담당 서비스
인증·권한	로그인, 역할별 API 접근	Node/Express
차량·운행	차량 등록, 출발·도착지 설정, 운행 시작·종료	Node + PostgreSQL/PostGIS
위치·경로	BIMS/단말 위치, 차량 제약을 반영한 A*	Routing/Tracking
영상·AI	Android 영상, 객체 박스·분할·거리 표시	Media Relay + Vision
녹화·재생	영상 세그먼트, 탐지·GPS 시간 동기화	Relay + Node + MinIO + Vision
가상 배차	경로 미리보기, 배차 수락, 통제·우회	Node + Routing
운영	HTTPS, TURN, 서비스별 실행 환경	Nginx + Coturn + Compose

2.2 비기능 요구사항

- 업무 데이터는 Node 가 관리하고, 경로 계산·일시적 위치 추적은 Routing 이 담당한다.
- 영상·추론 처리는 업무 API 와 분리하고 큐를 제한하여 지연 누적을 줄인다.
- 사용자 역할과 내부 서비스 인증을 적용하고 외부 통신은 HTTPS 로 제공한다.
- 가상 운행은 실제 GPS·영상·녹화 데이터와 분리하며, 서버에서 상태를 관리한다.

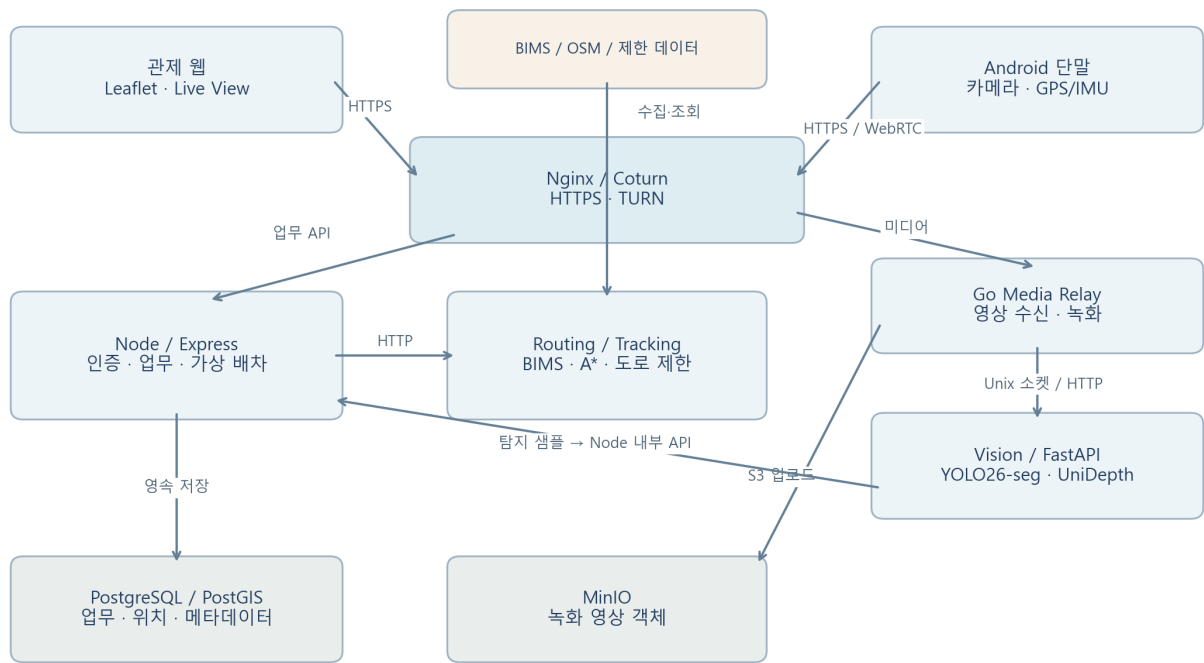
2.3 사용자 요구사항

운영자는 차량 선택, 경로 확인, Live View, 운행 기록 조회와 가상 배차를 한 화면에서 수행할 수 있어야 한다. 조회자는 읽기 전용 기능을 사용하며, Android 단말은 배정 운행을 받아 시작·종료하고 영상과 센서를 전송할 수 있어야 한다.

2.4 제약사항

도로 데이터의 적용 지역은 부산이며, 차량 프로파일은 시연을 위한 구성이다. Live View 는 한 번에 하나의 Android 스트림을 다룬다. GPU 추론과 실제 단말 송출은 해당 장비에서 검증한다. 실제 차량의 운행 중 경로 재배정은 이번 범위에서 제외한다.

3. 시스템 설계



영상·추론과 업무 처리를 서비스 경계에서 연결

그림 1. ITS 통합 시스템 구성

표 3. 계층별 구성과 책임

계층	구성요소	책임
클라이언트	관제 웹 / Android	지도·영상 표시 / 카메라·센서 송출
외부 접점	Nginx / Coturn	TLS 종단, 경로 라우팅, NAT 통과
업무 처리	Node / Express	인증·권한, 업무 API, 가상 시뮬레이션
계산	Routing / Tracking	도로 그래프, A*, BIMS·단말 위치 병합
미디어·비전	Go Relay / FastAPI Vision	영상 수신·녹화, 객체 분할·거리 추정
저장	PostgreSQL / PostGIS / MinIO	업무·공간 데이터, 영상 객체

서비스는 하나의 저장소에서 관리하되 런타임을 분리한다. Node 는 업무 상태와 저장을 담당하고 Routing 은 계산을 담당한다. Android 의 영상은 Go 릴레이를 통해 Vision 으로 전달하고, 녹화 객체는 MinIO 에 저장한다.

3. 시스템 설계 — 데이터와 AI 연동

3.1 데이터 설계

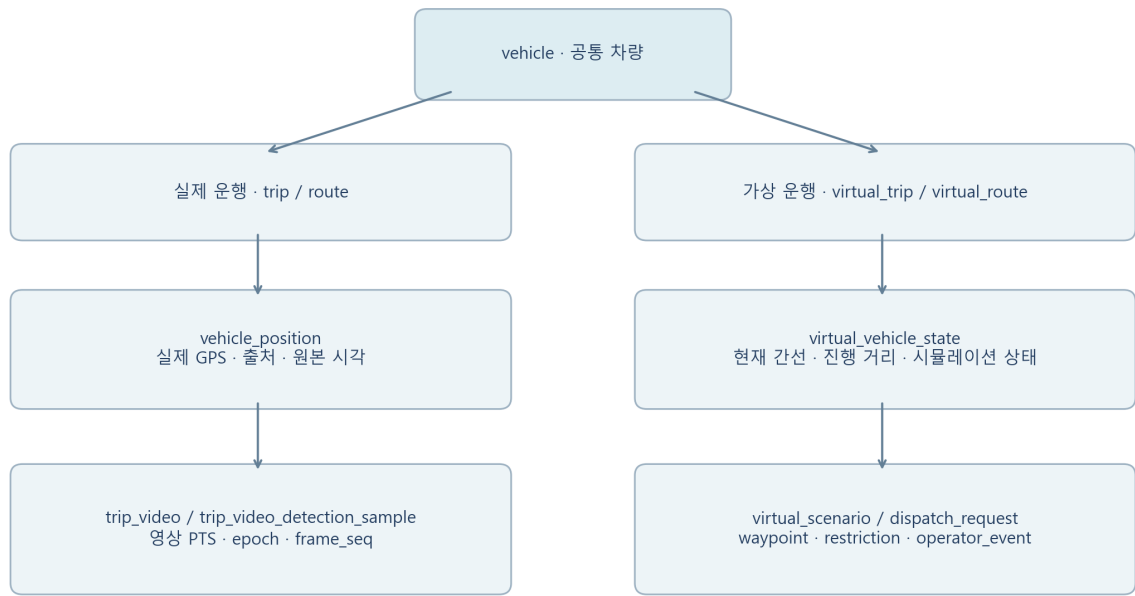


그림 2. 실제 운행·가상 배차 데이터 분리

실제 GPS 는 vehicle_position 에, 가상 위치는 virtual_vehicle_state 에 저장한다. 영상 세그먼트와 탐지 샘플은 원본 영상의 PTS, relay epoch 와 frame sequence 를 사용하여 연결한다. 같은 차량 테이블을 사용하더라도 출처와 운행 상태를 구분한다.

3.2 AI 모델 연동

표 4. 주요 AI 모델 역할

모델	역할	연동 결과
Custom YOLO26-seg	객체 탐지·인스턴스 분할	클래스·박스·객체 마스크
UniDepth	영상 기반 거리 측정	객체 거리 정보

분할 결과를 거리 정보와 연결하여 관제 화면에 제공한다. 객체 클래스별로 박스 또는 분할 표시를 선택할 수 있게 구성한다.

4. 개발 환경

표 5. 서비스별 개발 기술

영역	기술	용도
관제 웹	JavaScript, HTML/CSS, Leaflet	지도·워크스페이스·영상 연동
업무 백엔드	TypeScript, Express 5, Prisma 7	API, 인증, 업무 데이터 관리
경로·추적	Python, FastAPI, OSM 도로 그래프	차량 제한 판정·A*·위치 병합
미디어 릴레이	Go 1.22, Pion WebRTC v4	H.264 수신·전달·녹화
비전	Python 3.12, PyTorch, Ultralytics	분할·추적·거리 추정
차량 단말	Kotlin, CameraX, WebRTC, OkHttp	카메라·GPS/IMU·운행 제어
업무 저장소	PostgreSQL 17, PostGIS 3.5	업무·공간 데이터
객체 저장소	MinIO S3 API	녹화 영상 세그먼트
인프라	Docker Compose, Nginx, Coturn	컨테이너 실행·HTTPS-TURN

4.1 개발·관리 도구

Git 으로 코드를 관리하고, Node 는 npm, Python 서비스는 uv, 미디어 릴레이는 Go module, Android 는 Gradle 로 의존성을 관리한다. API 계약은 OpenAPI 로 확인하고, 데이터베이스 변경은 Prisma 마이그레이션과 ERD 문서에 반영한다.

4.2 모델 개발 환경

표 6. 분할 모델 학습 계획 기준

항목	기준
기반 모델	YOLO26s-seg
구조 실험	A0 기본 / A1 CARAFE / A2 ASPP / A3 CARAFE+ASPP / A4 디코더 결합
데이터셋	COCO 2017 인스턴스 분할
기본 입력 크기	640 × 640
비교 기준	Mask Precision-Recall·mAP50·mAP50-95
실험 기록	학습 설정, CSV, 가중치, 평가 그래프

4. 개발 환경 — UniDepth 모델 작성란

4.3 거리 측정 모델

UniDepth 를 거리 측정 모델로 사용한다. 상세 모델 자료는 아래 표에 추가한다.

표 7. UniDepth 모델 구성·학습 설정

항목	내용
모델명 / 버전	
모델 구조	
입력 / 출력	
데이터셋	
학습·검증 분할	
입력 해상도	
학습 epoch	
배치 크기	
Optimizer / 학습률	
가중치 / 실행 환경	

UniDepth 모델 구조도·처리 흐름 작성 공간

5. 개발 계획

표 8. 영역별 개발 계획

단계	Frontend	Backend	AI
요구사항·설계	관제 화면·사용자 흐름 정리	서비스 책임·API·DB 설계	데이터셋·클래스·평가 기준 정리
기반 구현	로그인·지도·차량 선택	인증·차량·운행 API, BIMS 연동	기본 분할 모델과 추론 연결
영상·기록	Live View·분할 표시·재생 UI	WebRTC 릴레이·녹화·메타데이터	CARAFE·ASPP·디코더 실험, 거리 연동
경로·배차	경로 미리보기·도로 통제 UI	차량 제약 A*, 가상 배차·시뮬레이션	교통 객체 성능 비교·추론 최적화
검증·정리	시연 흐름·오류 상태 확인	통합·권한·기록 검증	성능 표·그래프·최종 문서 정리

5.1 일정 및 담당

표 9. 일정·담당자 작성란

작업	기간	담당자
요구사항·설계		
관제·경로·배차		
영상·녹화·재생		
분할 모델 개발·평가		
거리 모델 정리·검증		
통합 검증·시연		

5.2 검증 계획

역할별 접근 제어, 경로 제한 반영, GPS 소스 전환, 영상·탐지 동기화, 가상 배차 수락과 차단 도로 대응을 확인한다. 분할 모델은 같은 평가 조건에서 비교하고, 거리 측정은 정답 거리 자료를 확보한 후 평가한다.

6. 시연 시나리오

6.1 차량 관제 및 영상 분석

1. 운영자가 로그인한 뒤 지도에서 차량을 선택한다.
2. 차량 위치와 계획 경로를 확인하고 Live View 를 연다.
3. Android 에서 배정 운행을 시작하고 카메라·GPS/IMU 를 송출한다.
4. 객체 박스·분할·거리 표시를 확인하고 클래스별 표시를 바꾼다.
5. GPS 가 지연되는 상황에서 지연 상태와 선택적 기록 보정 동작을 확인한다.

6.2 가상 배차 및 도로 통제

1. 가상 시나리오에서 차량·출발지·목적지·경유지를 선택한다.
2. 경로를 미리 보고 모의 운전자에게 배차를 요청한다.
3. 수락 후 차량이 이동하는 모습을 확인한다.
4. 정체 또는 차단 구역을 그리고 차량별 자동 추종 설정에 따른 대응을 확인한다.
5. 경유지를 변경하고 현재 위치에서 다시 계산한 경로를 확인한다.

6.3 운행 기록 재생

1. 종료한 운행의 녹화 목록을 선택한다.
2. 녹화 영상을 재생하고 해당 시점의 탐지 결과와 GPS 를 확인한다.
3. 타임라인을 이동하며 영상·지도·탐지 결과가 함께 갱신되는지 확인한다.

7. 결론

7.1 프로젝트 요약

본 프로젝트는 차량의 위치와 경로, 카메라 영상과 AI 분석, 운행 기록을 하나의 관제 흐름으로 연결한다. Node 의 업무 데이터 관리와 Python 의 경로·비전 계산, Go 의 미디어 처리를 분리하여 각 서비스의 역할을 명확히 한다.

Custom YOLO26-seg 는 CARAFE, ASPP 와 DeepLabV3+ 방식의 디코더를 실험하여 객체 분할 성능을 비교한다. UniDepth 는 거리 측정을 담당하며 상세 구성과 평가 결과는 해당 표에 보완한다. 가상 배차는 실제 운행과 분리된 환경에서 도로 통제와 우회 대응을 검토하는 기능으로 구성한다.

7.2 향후 개선 방향

- 부산 실제 주행 영상과 작은 객체·가림·야간 장면을 확보하여 분할 성능을 보완한다.
- 카메라 보정과 정답 거리 자료를 정리하고 거리 측정의 오차를 평가한다.
- 지도·영상·탐지의 동기화와 장시간 송출·녹화 안정성을 검증한다.
- 도로 제한 데이터의 정확도·갱신 체계를 개선하고 실제 운행 적용 가능성을 검토한다.

7.3 작성 근거

시스템 구성: P4 통합 백엔드 시스템 설계 문서 및 현재 저장소. 분할 모델 구성: 제공된 yolo_carafe_aspp 소스와 실험 설정. 이전 프로젝트 PDF 는 문서 구성과 서식 참고에 사용하였다.